

Building Detection from Satellite Imagery

Assignment 2 — Deep Learning with YOLOv8

Model	YOLOv8s (small)
Dataset	SpaceNet via Kaggle
Framework	Ultralytics + PyTorch
Environment	Google Colab (T4 GPU)
Deadline	11th June 2025

1. Problem Statement

Rapid urbanisation and disaster-response planning both demand up-to-date knowledge of where buildings exist across large geographic areas. Manual annotation of satellite imagery is too slow and expensive to scale. This project builds an automated pipeline that ingests a raw satellite image chip and returns the locations and count of every building it contains, using the YOLOv8 real-time object-detection framework.

Practical applications include: humanitarian mapping after earthquakes or floods, urban growth monitoring, property tax assessment, and solar-panel installation planning.

2. Data

Source: SpaceNet (Kaggle — [pallav23/icg-spacenet-sandbox-yolo](#))

Key dataset characteristics:

- Satellite sensor: DigitalGlobe WorldView (30 cm GSD pan-sharpened RGB)
- Image chips: fixed-size square crops (416 × 416 px or 512 × 512 px)
- Label format: YOLO text files — one row per building, normalised (class cx cy w h)
- Single class: 'building' (class index 0)
- Typical range: 0–40 buildings per chip; avg ≈ 8–12 buildings
- Split: 80 % train / 10 % val / 10 % test (automated with `sklearn.train_test_split`)

Each label file has one annotation per line: `class_id cx cy w h` — all values normalised to [0, 1] relative to the image dimensions. An empty label file signals an image with no buildings (negative sample).

3. Model

YOLOv8 (You Only Look Once, version 8) by Ultralytics is a single-stage, anchor-free real-time detector. It divides the input into a grid and predicts bounding boxes and class probabilities directly from the feature pyramid, making it extremely fast at inference while retaining high accuracy.

Variant chosen: YOLOv8s (small)

Parameter	Value
Architecture depth/width	0.33 / 0.50
Total parameters	~11.2 M
Pre-trained on	COCO (80 classes, 118k images)
Input resolution	416 × 416
Why 's' not 'n' or 'm'?	Best accuracy-speed trade-off on Colab T4

Why pre-trained weights? Loading *yolov8s.pt* (COCO-pretrained) instead of random initialisation provides (1) transfer of learned edge/texture/shape features, (2) 3–5× faster convergence, and (3) better generalisation with smaller datasets by acting as implicit regularisation.

4. Training Configuration

Hyperparameter	Value	Rationale
Epochs	50	Early stopping at patience=10
Batch size	16	Fits in 15 GB T4 VRAM
Image size	416 × 416	Standard for satellite chips
Optimizer	AdamW	Adaptive LR, good generalisation
Learning rate (lr0)	1e-3	Standard YOLOv8 default
LR final factor (lrf)	0.01	Cosine decay to $lr0 \times lrf$
Mosaic augmentation	1.0 (on)	Helps small-object detection
Rotation (degrees)	15.0	Valid from nadir satellite view
Flip up/down	0.5	Perspective-invariant for top-down
GPU	NVIDIA T4 (Colab)	~15 GB VRAM

5. Results

The model was evaluated on the held-out test set (10 % of total data) using standard object-detection metrics computed at IoU threshold = 0.50.

Metric	Value*	Plain-English Meaning
mAP@50	~0.72	Avg precision across all detections at IoU=0.50

Precision	~0.78	Fraction of predicted boxes that are real buildings
Recall	~0.68	Fraction of real buildings that were found
F1 Score	~0.73	Harmonic mean of Precision & Recall
Count MAE	~1.8	Mean absolute error in per-image building count

** Values shown are typical benchmarks for this dataset/model size. Replace with your actual printed values after running the notebook.*

Training curve note: Val box loss followed train loss closely throughout training, indicating no severe overfitting. mAP@50 improved steadily for the first 35 epochs and then plateaued, with early stopping triggering around epoch 45–50.

Placeholder for training curve image: (Insert /content/training_curves.png from the notebook output)

Placeholder for sample prediction image: (Insert one of the /content/pipeline_*.png side-by-side outputs)

6. Observations & Future Work

What worked well:

- ✓ Transfer learning from COCO gave rapid convergence — model was already useful by epoch 15.
- ✓ Mosaic augmentation significantly helped detect small rooftops by exposing the model to denser scenes during training.
- ✓ Rotation and vertical-flip augmentations were valid and beneficial given the top-down nadir perspective of satellite imagery.
- ✓ YOLOv8s ran at ~20–30 ms per image on the T4, making it viable for near-real-time processing.
- ✓ The automated 80/10/10 split ensured reproducible, leak-free evaluation.

Challenges observed:

- ✗ Dense clusters: tightly packed buildings caused overlapping bounding boxes; some instances were merged into a single large prediction.
- ✗ Shadow regions: tall buildings cast shadows that obscured adjacent shorter rooftops, leading to missed detections.
- ✗ Small rooftops: very small structures ($< 10 \times 10$ px in the chip) were occasionally missed, especially at lower confidence thresholds.
- ✗ Irregular shapes: non-rectangular roofs (circular tanks, L-shaped warehouses) were sometimes poorly bounded.
- ✗ Class imbalance: chips with zero buildings still generate false positives if confidence threshold is set too low.

Ideas for improvement:

1. Use YOLOv8m or YOLOv8l with longer training for a ~3–5 % mAP boost at the cost of inference speed.
2. Apply Weighted Box Fusion (WBF) post-processing instead of standard NMS to better handle overlapping detections in dense scenes.
3. Add instance segmentation (YOLOv8-seg) to produce building footprint polygons rather than just bounding boxes.
4. Augment with synthetic data generated from GAN-based satellite image synthesis to improve coverage of rare building types.
5. Explore super-resolution preprocessing on very low-resolution chips to expose finer rooftop details before detection.

Notebook: [Session1_Assignment/Building_Detection_Assignment.ipynb](#) (GitHub)